

Ciência de Dados para Pesquisadores da Saúde

Módulo 1: Ferramentas, linguagens e possibilidades *Projeto Saúde Data Science*

1. Sobre o Curso

Objetivo

Introduzir estudantes de iniciação científica, pós-graduação e pesquisadores ao uso da ciência de dados na pesquisa em saúde.

Módulos

- Módulo 1: Ferramentas, linguagens e possibilidades (12h)
- Módulo 2: Manipulação de dados, visualização e bioestatística com o R (12h) (Set)
- Módulo 3: Machine learning na saúde (12h) (Nov)
- Módulo 4: Projeto de aplicação (4h)

Conteúdo Programático do Módulo 1

1. Introdução à Ciência de Dados (Teoria)
2. Introdução ao uso do R e RStudio (Prática)
3. Fundamentos da linguagem R (Teoria)
4. Controle de versões Git/GitHub (Prática)
5. Criação de apresentações, relatórios, web-sites, artigos, teses, dashboards e aplicativos web (Prática)

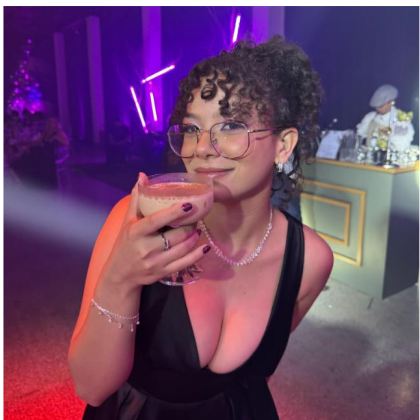
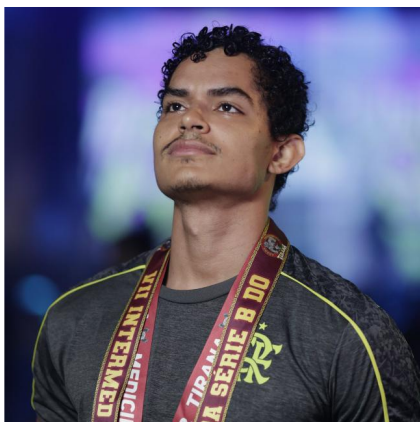
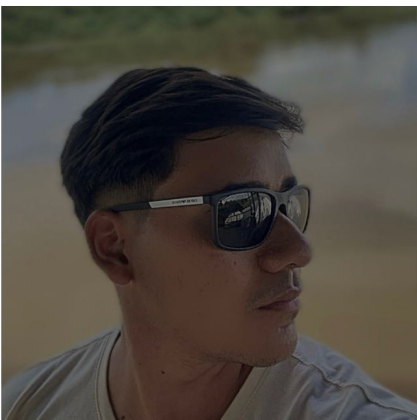
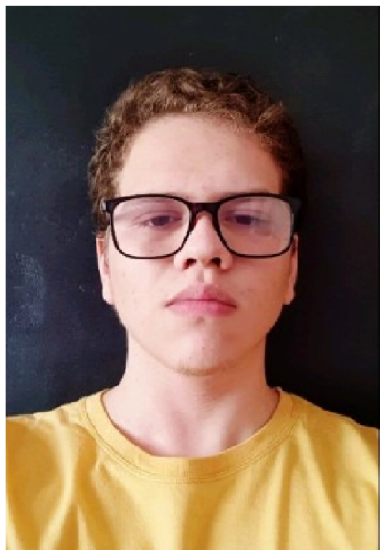
Projeto Saúde Data Science

Objetivo

Criar oportunidades de interação dialógica com a sociedade por meio da elaboração de produtos de dados associados à ciência de dados.

Equipe do Projeto *Saúde Data Science*

Projeto de Extensão UFR N° 696/2025



2. Introdução

O que é ciência de dados (*Data Science*)

Campo acadêmico interdisciplinar que utiliza estatística, computação científica, métodos científicos, processos, algoritmos e sistemas para extrair ou extrapolar conhecimento e insights de **dados ruidosos, estruturados e não estruturados** (Dhar, 2013).

Estruturados

```
knitr::include_graphics("dados_sinan.png")
```

Arquivo

Página Inicial

Inserir

Layout da Página

Fórmulas

Dados

Revisão

Exibir

Ajuda

Colar

Calibri

11

A[^]

A_v

N. Estruturados

```
knitr::include_graphics("dados_ficha_notificação.png")
```

República Federativa do Brasil
Ministério da Saúde

SINAN
SISTEMA DE INFORMAÇÃO DE AGRAVOS DE NOTIFICAÇÃO
FICHA DE NOTIFICAÇÃO

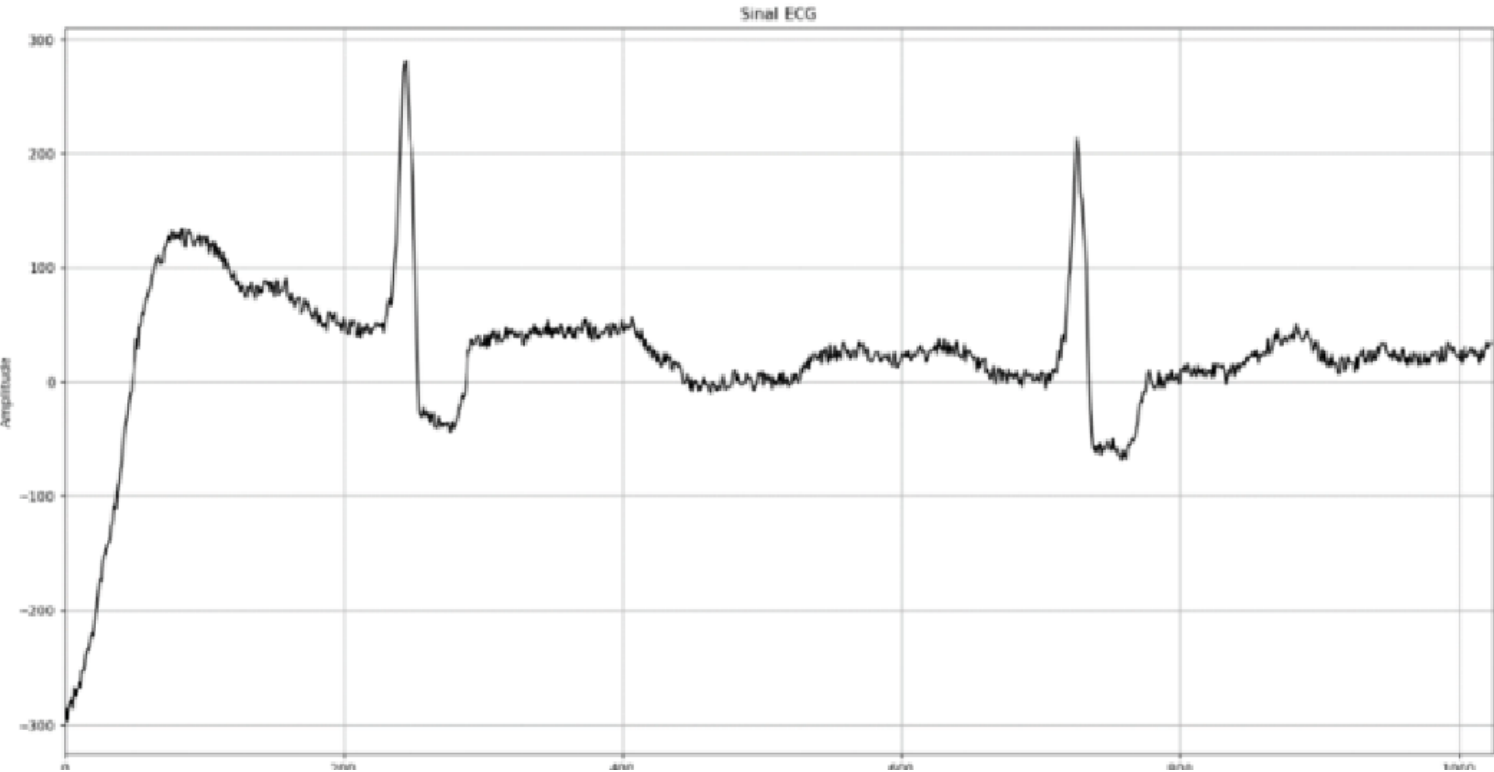
Nº

Dados Gerais	1 Tipo de Notificação 1 - Negativa 2 - Individual 3 - Surto	2 Data da Notificação
	3 Município de Notificação	Código (IBGE)
Notificação Individual	4 Unidade de Saúde (ou outra fonte notificadora)	Código
	5 Agravado/doença	Código (CID10)
	6 Data dos Primeiros Sintomas	7 Nome do Paciente
	8 Data de Nascimento	9 (ou) Idade D - dias M - meses A - anos
	10 Sexo M - Masculino F - Feminino I - Ignorado	11 Raça/Cor 1-Branca 2-Preta 3-Amarela 4-Parda 5-Indígena 9-Ignorado
Notificação de Surto	12 Escolaridade (em anos de estudo concluídos) 1-Nenhuma 2-De 1 a 3 3-De 4 a 7 4-De 8 a 11 5-De 12 e mais 6-Não se aplica 9-Ignorado	13 Número do Cartão SUS
	14 Nome da mãe	15 Agravado/doença
	Código (CID10)	16 Data dos 1ºs Sintomas do 1º Caso Suspeito
	17 Nº de Casos Suspeitos	18 Local Inicial de Ocorrência do Surto 1 - Casos Restritos a uma Mesma Família 2 - Casos Restritos a uma Escola/Creche 3 - Casos Restritos a um Hospital 4 - Casos Restritos a outra Instituição (alojamento, asilo, trabalho) 5 - Casos Dispersos no Bairro 6 - Casos Dispersos Pelo Município 7 - Casos Dispersos em mais de um Município 8 - Outros
Endereço	19 Logradouro (rua, avenida,...)	Código
	20 Número	21 Complemento (apto., casa, ...)
	22 Ponto de Referência	23 UF

17	2024-03-20	2024	3	20	quarta-feira	12	51	510760	1579	986763
18	2024-03-22	2024	3	22	sexta-feira	12	51	510760	1579	986763

Ruidosos

```
knitr::include_graphics("dados_ruidosos.png")
```



A ciência de dados está relacionada com

- Big Data

- **Mineração de Dados (*Data Mining*)**
- **Aprendizagem de Máquina (*Machine Learning*)**

Mineração de Dados (*Data Mining*)

“É o processo de extrair e descobrir padrões em grandes bancos de dados envolvendo métodos relacionados à aprendizagem de máquina, estatística e sistemas de bancos de dados.” Wikipedia

É o processo de descobrir automaticamente informação útil em grandes repositórios de dados (Tan et al., 2005)

Histórico do *Data Mining*

- **Década de 1960-70:** Surgimento dos primeiros bancos de dados e linguagens de consulta (ex: SQL).
- **Década de 1980:** Avanços em armazenamento e recuperação de dados; algoritmos de aprendizado simbólico.
- **Década de 1990:** Popularização do termo *data mining*; surgimento de ferramentas comerciais (ex: IBM Intelligent Miner, SAS Enterprise Miner).
- **2000s:** Integração com *big data*, algoritmos escaláveis, mineração de dados em tempo real.
- **2010s em diante:** Consolidação como parte da ciência de dados; uso intensivo de aprendizado de máquina e redes neurais.

Aprendizagem de Máquina (*Machine Learning*)

- Consiste em desenvolver algoritmos que permitem os computadores aprenderem dos dados e tomarem decisões baseadas em dados
- Surgiu na década de 1950
- **Arthur Samuel (1959)** definiu *machine learning* como:
“o campo de estudo que dá aos computadores a habilidade de aprender sem serem explicitamente programados.”

Aspecto	Mineração de Dados	Aprendizagem de Máquina
Objetivo principal	Extrair padrões úteis e conhecimento de dados	Fazer previsões ou tomar decisões com base em dados
Enfoque	Exploração e descoberta de padrões em grandes conjuntos de dados	Generalização para novos dados a partir de exemplos passados
Tarefa típica	Agrupamento, associação, descoberta de regras	Classificação, regressão, clustering supervisionado
Motivação original	Análise de grandes volumes de dados corporativos	Construção de algoritmos que aprendem automaticamente
Uso comum em	Negócios, marketing, banco de dados	Inteligência artificial, robótica, sistemas autônomos
Base técnica	Estatística + banco de dados + ML	Estatística + algoritmos + computação

Por que Ciência de Dados na Pesquisa em Saúde?

O processo da Ciência de Dados é muito semelhante ao da pesquisa quantitativa

As mudanças tecnológicas que iniciaram na década de 50, e se intensificaram em 1990 e em 2020, mudaram o material empírico e o fazer científico.

► Ethn Dis. 2017 Apr 20;27(2):95–106. doi: [10.18865/ed.27.2.95](https://doi.org/10.18865/ed.27.2.95) 

Big Data Science: Opportunities and Challenges to Address Minority Health and Health Disparities in the 21st Century

[Xinzhi Zhang](#)^{1,✉}, [Eliseo J Pérez-Stable](#)¹, [Philip E Bourne](#)², [Emmanuel Peprah](#)³, [O Kenrik Duru](#)⁴, [Nancy Breen](#)¹, [David Berrigan](#)⁵, [Fred Wood](#)⁶, [James S Jackson](#)⁷, [David WS Wong](#)⁸, [Joshua Denny](#)⁹

► [Author information](#) ► [Article notes](#) ► [Copyright and License information](#)



The Science of Data About People

► Int J Popul Data Sci. 2019 Nov 20;4(2):1143. doi: [10.23889/ijpds.v4i2.1143](https://doi.org/10.23889/ijpds.v4i2.1143) 

Data Centre Profile: The Provincial Health Data Centre of the Western Cape Province, South Africa

[A Boulle](#)^{1,2,*}, [A Heekes](#)^{1,2}, [N Tiffin](#)^{1,2}, [M Smith](#)^{1,2}, [T Mutemaringa](#)^{1,2}, [N Zinyakatira](#)^{1,2}, [F Phelanyane](#)^{1,2}, [C Pienaar](#)¹, [K Buddiga](#)^{1,2}, [E Coetzee](#)^{1,2}, [R van Rooyen](#)^{1,2}, [R Dyers](#)^{1,3}, [N Fredericks](#)¹, [A Loff](#)¹, [L Shand](#)¹, [M Moodley](#)^{1,2}, [I de Vega](#)¹, [K Vallabhjee](#)^{1,2}

► [Author information](#) ► [Article notes](#) ► [Copyright and License information](#)

PMCID: PMC7482518 PMID: [32935043](https://pubmed.ncbi.nlm.nih.gov/32935043/)

 Journal of Medical Internet Research



Journal Information ▼

Browse Journal ▼

Published on 25.11.2024 in Vol 26 (2024)

 Preprints (earlier versions) of this paper are available at <https://preprints.jmir.org/preprint/55897>, first published December 30, 2023.



Implementation of Machine Learning Applications in Health Care Organizations: Systematic Review of Empirical Studies

[Luigi M Preti](#)¹ ; [Vittoria Ardito](#)¹ ; [Amelia Compagni](#)^{1,2} ; [Francesco Petracca](#)¹ ; [Giulia Cappellaro](#)^{1,2} 

Termos de busca (google scholar)	Resultados
“machine learning” e “health”	4.800.000
“data mining” e “health”	5.440.000
“data science” e “health”	7.270.000

- Em geral, o uso da ciência de dados na pesquisa:
 - amplia o alcance
 - aumenta o rigor
 - eleva o impacto real

Ciência de Dados na Pesquisa: repensar o fazer científico

1. Implicações da era do Big Data
2. Implicações dos desenvolvimentos em ML, DM e IA
3. Implicações do uso de linguagem de programação
4. Efeitos potenciais na pesquisa universitária

Ciência de Dados na Pesquisa: repensar o fazer científico

Big Data e a revolução dos dados

- Dados administrativos se tornaram dados secundários de pesquisa
 - Ex: Registros eletrônicos do **e-SUS**, **SIH/SUS**, **SINAN** e **SIVEP**
- Dados não usuais tornaram-se mais acessíveis
 - Ex: prescrições médicas, áudios de consultas, imagens de exames, etc.
- Dispositivos e sistemas passaram a gerar outros dados
 - Ex: Dados oriundos de dispositivos vestíveis (wearables) e apps de saúde

Machine Learning, Data Mining e Inteligência Artificial

- Novas e melhores ferramentas de análise
 - Análise de dados não estruturados
 - * Ex: textos, imagens, vídeos
 - Mineração
 - Predição
 - * Ex: Predição da mortalidade hospitalar
 - Classificação
 - * Ex: Algoritmo de classificação baseado em deep learning para detectar retinopatia diabética
 - Agrupamento
 - * Ex: Identificar subgrupos de pacientes com insuficiência cardíaca congestiva com base em características clínicas, visando apoiar tratamentos personalizados

Uso de uma linguagem de programação

- Aumenta a **reprodutibilidade** e **transparência** (exemplo: github)
- Melhora a **comunicação dos resultados** (exemplo: preliminar e desenho)
- Melhora o diálogo entre grupos de pesquisa (exemplo: [aplicativo web](#))
- Melhora o fluxo de orientação/mentoria
- Fortalece a **ciência aberta** e a **confiança pública**
- Viabiliza formação interdisciplinar
- Permite interação mais natural com ferramentas de IA (exemplo: chatgpt)

Uso da IA como assistente de pesquisa

- Aceleração da codificação
 - Geração rápida de funções e scripts com base em linguagem natural
 - Sugestões otimizadas de código para evitar erros
- Melhora na organização do fluxo de análise de dados
 - Visualizações exploratórias automatizadas
 - Criação de pipelines de pré-processamento

Uso da IA como assistente de pesquisa

- Auxilia na documentação e reprodutibilidade
 - Comentários automáticos e claros no código
 - Templates para RMarkdown, Quarto, Jupyter etc.
- Auxílio em análises estatísticas e modelagem
 - Sugestões e explicações de testes estatísticos
 - Comparação de modelos
 - Auxilia na construção de simulações
 - Ajuda a encontrar erros no código (debug)

Uso da IA como assistente de pesquisa

- Auxilia na divulgação dos resultados
 - Templates para formatação de monografias, dissertações e teses
 - Eloboração de produtos de dados
- Pode acelerar o aprendizado
 - Explicações sob demanda de sintaxe e conceitos estatísticos
 - Referência a pacotes, funções e boas práticas
 - Respostas a perguntas avançadas de modelagem e dados

Uso da IA como assistente de pesquisa

Fortalecimento da relação entre Pesquisa, Extensão e Inovação

- Novas possibilidade de pesquisas com aplicações práticas
- Análise de dados em tempo real
- Criação de produtos tecnológicos (produtos de dados)
- Maior protagonismo do estudante
- Oportunidade de formação interprofissional
- Maior empregabilidade

Fortalecimento da relação entre Pesquisa, Extensão e Inovação

Por que a linguagem/software R

- O R é uma linguagem de programação *funcional*
- As funções são armazenadas em pacotes
 - Pacotes básicos + pacotes instaláveis
 - Mais de 16.000 pacotes (2020)
- O software e a linguagem *gratuitos*
- Pode ser utilizado em diversos sistemas operacionais

Por que a linguagem/software R

- Possui uma comunidade de colaboração ativa
- Possui um sistema de ‘controle de qualidade’
- É a linguagem preferida dos estatísticos
- É uma das linguagens de análise estatísticas mais utilizadas no mundo
- Viabiliza o trabalho IA integrada

Por que a linguagem/software R

3. Usando o R!

Prática: Baixar e instalar o R

<https://cran.r-project.org/bin/windows/base/>



[Home]

Download

CRAN

R Project

About R

Logo

Contributors

What's New?

Reporting Bugs

Conferences

Search

Get Involved: Mailing Lists

Get Involved: Contributing

Developer Pages

R Blog

R Foundation

Foundation

Board

Members

Donors

Donate

Help With R

Getting Help

Documentation

Manuals

FAQs

The R Project for Statistical Computing

Getting Started

R is a free software environment for statistical computing and graphics. It compiles and runs on a wide variety of UNIX platforms, Windows and MacOS. To [download R](#), please choose your preferred [CRAN mirror](#).

If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

News

- [R version 4.4.2 \(Pile of Leaves\)](#) has been released on 2024-10-31.
- We are deeply sorry to announce that our friend and colleague Friedrich (Fritz) Leisch has died. [Read our tribute to Fritz here](#).
- [R version 4.3.3 \(Angel Food Cake\)](#) (wrap-up of 4.3.x) was released on 2024-02-29.
- [Registration for useR! 2024](#) has opened with early bird deadline March 31 2024.
- You can support the R Foundation with a renewable subscription as a [supporting member](#).

News via Mastodon



R_Foundation
R-4.4.2 (codename "Pile of Leaves") was released this morning.

This is a minor update, with a few bug fixes: [stat.ethz.ch/R-manual/R-patche...](#)

You can get the source code from CRAN: [cloud.r-project.org](#)

Binaries for various platforms will appear in due course.
(The Mac binaries will be delayed for a couple of days because the maintainer is travelling.)

Oct 31, 2024



R_Contributors
Join us at the online R Dev Day @ LatinR 2024!

Ativar o Windows
Acesse Configurações para ativar o Windows.



CRAN Mirrors

The Comprehensive R Archive Network is available at the following URLs, please choose a location close to you. Some statistics on the status of the mirrors can be found here: [main page](#), [windows release](#), [windows old release](#).

If you want to host a new mirror at your institution, please have a look at the [CRAN Mirror HOWTO](#).

0-Cloud

<https://cloud.r-project.org/>

Argentina

<http://mirror.fcaglp.unlp.edu.ar/CRAN/>

Australia

<https://cran.csiro.au/>
<https://mirror.aarnet.edu.au/pub/CRAN/>
<https://cran.ms.unimelb.edu.au/>

Austria

<https://cran.wu.ac.at/>
<https://mirror.honeybet.de/>

Belgium

<https://www.freeststatistics.org/cran/>
<https://ftp.belnet.be/mirror/CRAN/>

Brazil

<https://cran-r.c3sl.ufpr.br/>
<https://vps.fmvz.usp.br/CRAN/>
<https://brieger.esalq.usp.br/CRAN/>

Bulgaria

<https://ftp.uni-sofia.bg/CRAN/>

Canada

<https://muug.ca/mirror/cran/>
<https://mirror.csclub.uwaterloo.ca/CRAN/>
<https://cran-mirror.rafael.ca/>

Chile

<https://cran.dcc.uchile.cl/>

China

<https://mirrors.tuna.tsinghua.edu.cn/CRAN/>
<https://mirrors.bfsu.edu.cn/CRAN/>
<https://mirrors.pku.edu.cn/CRAN/>

Automatic redirection to servers worldwide, currently sponsored by Posit

Universidad Nacional de La Plata

CSIRO
AARNET
School of Mathematics and Statistics, University of Melbourne

Wirtschaftsuniversität Wien
easyname.at

Patrick Wessa
Belnet, the Belgian research and education network

Universidade Federal do Paraná
University of Sao Paulo, Sao Paulo
University of Sao Paulo, Piracicaba

Sofia University

Manitoba Unix User Group
University of Waterloo
Rafal Rzeczkowski

Departamento de Ciencias de la Computación, Universidad de Chile

TUNA Team, Tsinghua University
Beijing Foreign Studies University
Peking University

Ativar o Windows
Acesse Configurações para ativar o Windows.

Caixa d... x Introdu... x Pessoal x HORAR x SEI - UF x (9) Wh... x Meus c... x SEI/UF... x séries t... x r softw... x Economi... x The Co... x

cran-r.c3slufpr.br

Personalizar e controlar o Google Chrome

R

CRAN
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

About R
[R Homepage](#)
[The R Journal](#)

Software
[R Sources](#)
[R Binaries](#)
[Packages](#)
[Task Views](#)
[Other](#)

Documentation
[Manuals](#)
[FAQs](#)
[Contributed](#)

Donations
[Donate](#)

R-4.4.1 for Windows

[Download R-4.4.1 for Windows](#) (82 megabytes, 64 bit)
[README on the Windows binary distribution](#)
[New features in this version](#)

This build requires UCRT, which is part of Windows since Windows 10 and Windows Server 2016. On older systems, UCRT has to be installed manually from [here](#).

If you want to double-check that the package you have downloaded matches the package distributed by CRAN, you can compare the [md5sum](#) of the .exe to the [fingerprint](#) on the master server.

Frequently asked questions

- [Does R run under my version of Windows?](#)
- [How do I update packages in my previous version of R?](#)

Please see the [R FAQ](#) for general information about R and the [R Windows FAQ](#) for Windows-specific information.

Other builds

- Patches to this release are incorporated in the [r-patched snapshot build](#).
- A build of the development version (which will eventually become the next major release of R) is available in the [r-devel snapshot build](#).
- [Previous releases](#)

Note to webmasters: A stable link which will redirect to the current Windows binary release is [<CRAN MIRROR>/bin/windows/base/release.html](#).

Last change: 2024-06-15

Ativar o Windows
Acesse Configurações para ativar o Windows.

Inserindo código diretamente no console do R

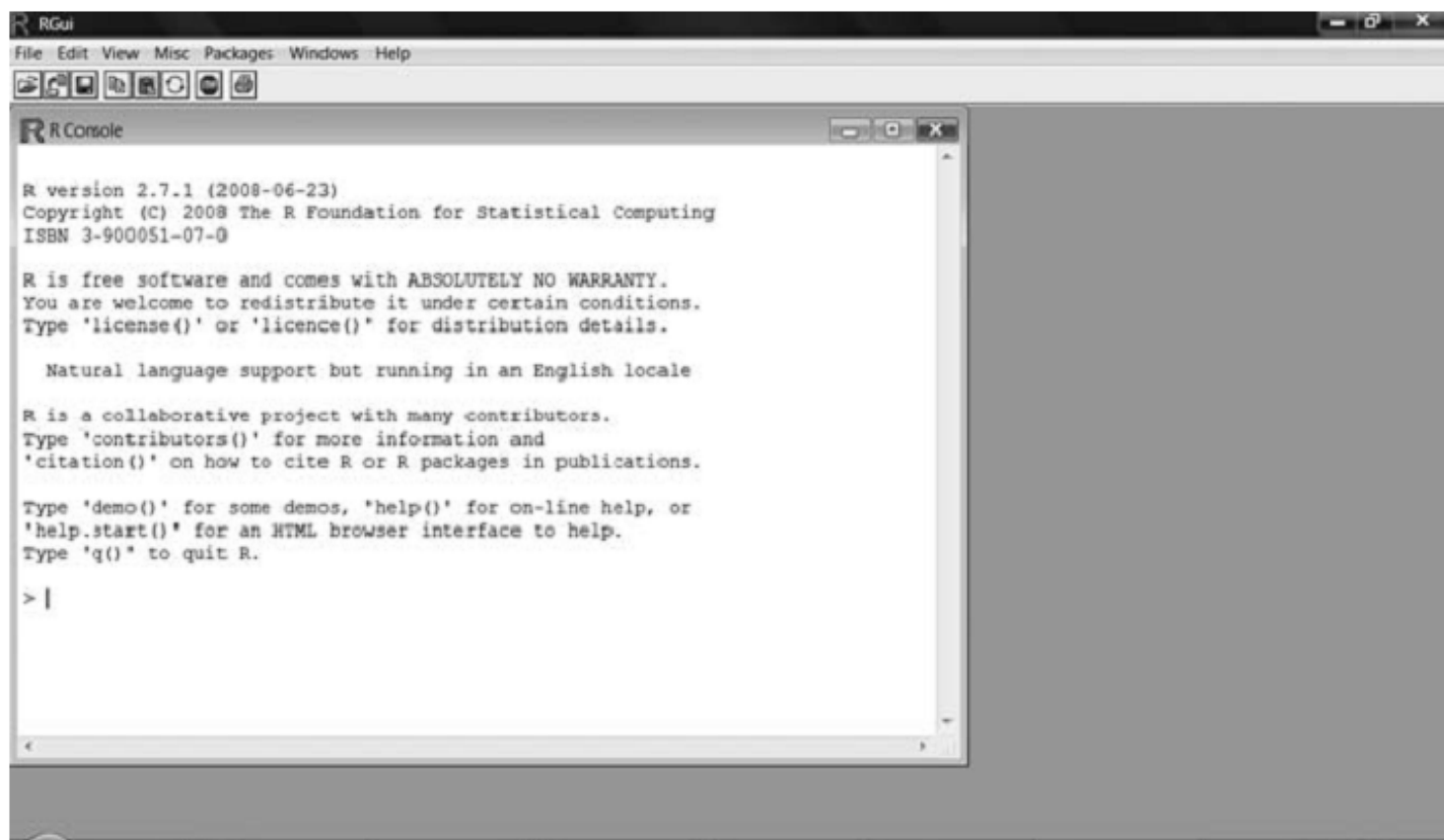
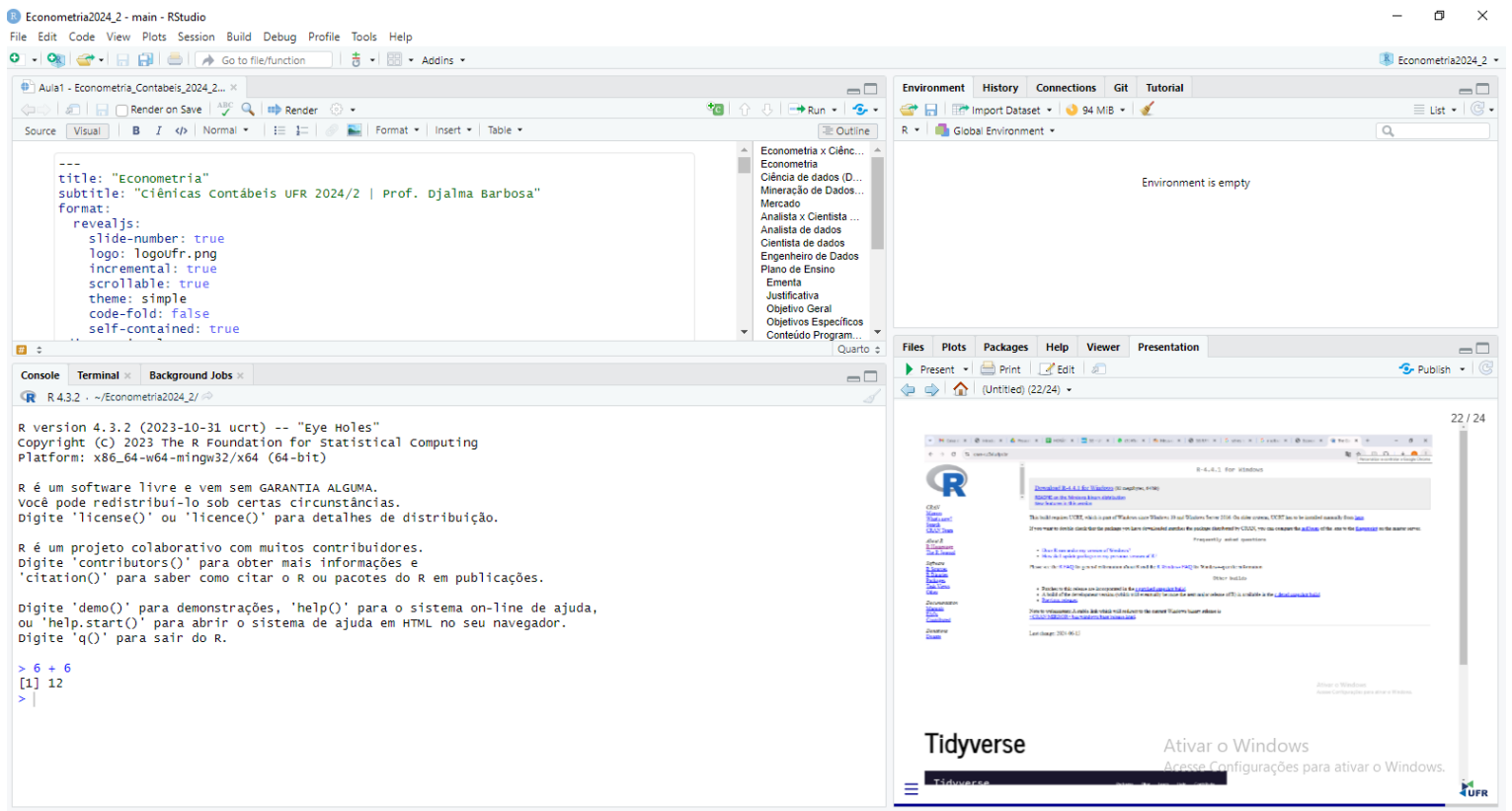


Figure 1: Console do R

Prática: baixar e instar o RStudio



Como usar o RStudio

1. Escrevendo no console
2. Utilizando seus tipos de arquivo
 - R Script (simples e útil)
 - RMarkdown
 - Quarto
3. Utilizando projetos
4. Integrando o GitHub
5. Com um co-piloto (IA)

Escrevendo no painel *console* do RStudio

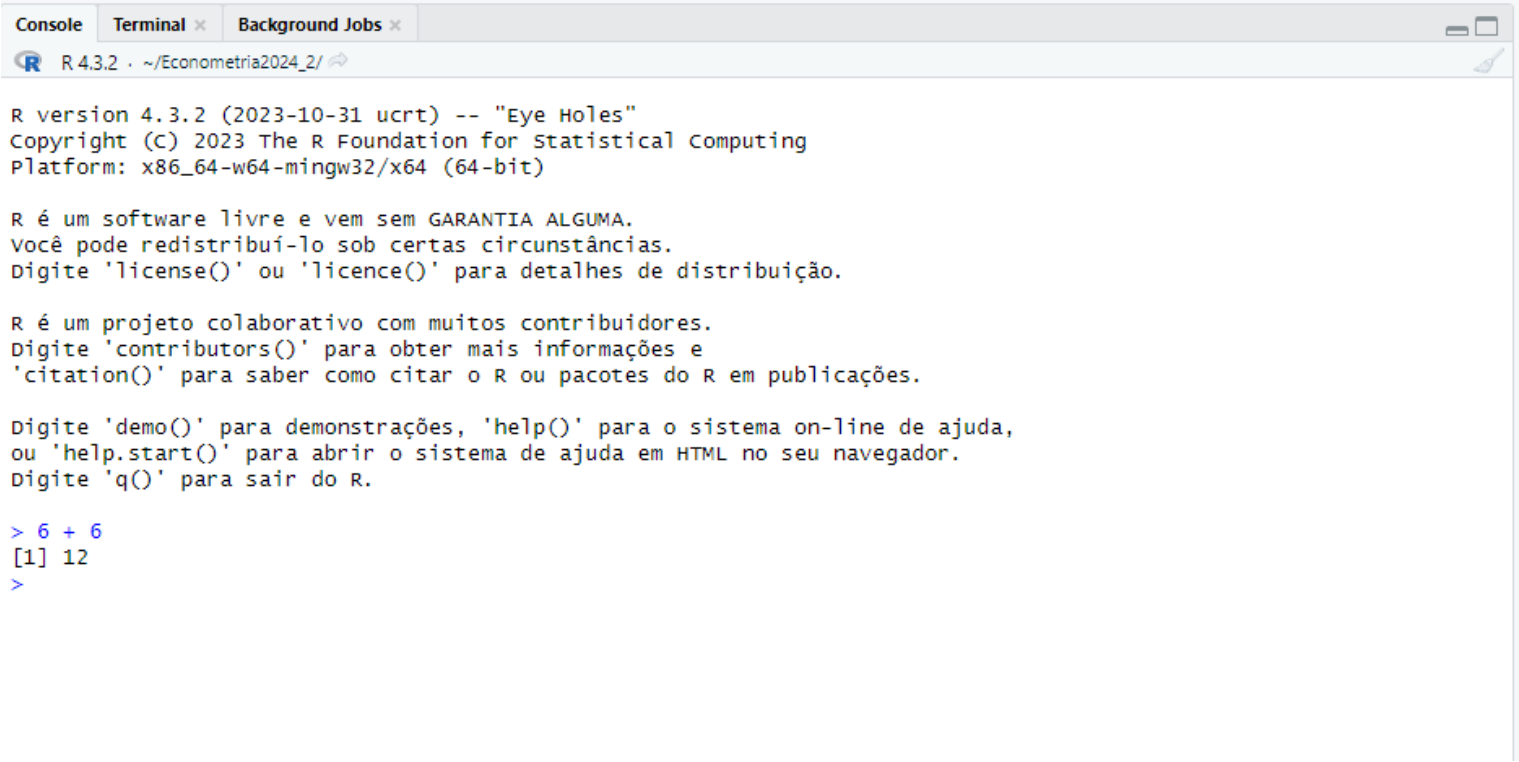


Figure 2: Console do RStudio

Utilizando *R scripts*

- Um *script* é um texto em forma de comandos (códigos), que o R interpreta como uma ação a ser executada.
- É um roteiro de execução
- Executa código sequencialmente
- É o arquivo utilizado pelo R, quando não se usa o RStudio

Escrevendo um *R script*

Prática

Clique em: *Menu > File > New File > R Script*
Escreva no seu R Script: 2+2
Coloque o cursor do mouse na linha com o código e clique em ‘Run’.
Para salvar: *File > Save*

Em geral os *scripts* possuem os seguintes elementos:

Termo	Sinônimos	Descrição
Base de dados	<i>dataframe</i> , Banco de dados	Estrutura que armazena dados
Variável	<i>Column</i> , Coluna	Campo que armazena valores para análise
Linha	<i>Row</i> , Observação, Registro	Cada entrada ou conjunto de valores
Função	Verbo, Comando	Ação que o R executa
Argumento	Parâmetro	Define opções dentro de uma função
Output	Resultado	Retorno gerado pelo código

Funções

Note

- O R é uma linguagem funcional
- Funções vêm associadas a parênteses (), onde colocamos os *argumentos da função*.
- Exemplos da matemática:
 - $y = f(x)$
 - $z = f(x, y)$... Latex
 - A função mais simples no R é a criação de um conjunto de valores (*vetor*)

Prática

```
# Soma 2 com 2
2+2

# Cria o vetor x
x <- c(1,1,2,3,5,8) # define o objeto x

mean(x) # Calcula a media de x
```

Utilizando pacotes (*packages*) Prática

- Instalando com ‘click’ *Tools, Install Packages*.
- Com comando:

```
install.packages("tidyr")
```

- Carregando o pacote instalado

```
library(tidyr)
```

– Você sempre precisará recarregar os novos pacotes após encerrar uma seção

- Verificando os pacotes instalados

```
sessionInfo()
```

- Atualizando pacotes

Tidyverse

Filosofia

```
knitr::include_graphics("tidyverse.png")
```



R packages for data science

The tidyverse is an opinionated **collection of R packages** designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

Pacotes do Tidyverse

```
knitr::include_graphics("pacotes_tidyverse.png")
```

Pacote	Descrição
ggplot2	Cria gráficos baseado na gramática dos gráficos.
tibble	Leitura e gravação de um moderno formato de banco de dados.
tidyr	Auxílio para arrumar e transformar banco de dados.
readr	Leitura e gravação de arquivos delimitados por vírgulas ou tabulação.
purrr	Otimização e desempenho de algoritmos.
dplyr	Manipulação e limpeza de dados.
stringr	Manipulação de texto.
forcats	Manipulação de fatores.

O *tidyverse* é um metapacote que segue uma filosofia de design, gramática e estruturas de dados em comum para que diversos pacotes úteis para a ciência de dados possam ser utilizados em conjunto

Prática

Vamos instalar e carregar os pacotes do *tidyverse*

```
if(!require(tidyverse)) install.packages("tidyverse")

library(tidyverse)

# Agora podemos usar uma função 'tibble' que faz parte do tidyverse

y <- c(1,2,3,4,5,6)

tibble(x,y) # retorna o objeto no console

dados <- tibble(x,y)
```

Dicas de organização do *script*

Bagunçado

```
library(dplyr)
dados=data.frame(nome=c("Ana","Bruno","Carlos","Daniela"),idade=c(23,35,31,28),salario=c(2500,4200,3900,3100))
dados_filtrados=dados%>%filter(salario>3000)
media_idade=mean(dados_filtrados$idade)
print(dados_filtrados)
```

Organizado

```
# Carrega pacotes necessários
library(dplyr)

# Cria um dataframe de exemplo
dados <- data.frame(
  nome = c("Ana", "Bruno", "Carlos", "Daniela"),
  idade = c(23, 35, 31, 28),
  salario = c(2500, 4200, 3900, 3100)
)

# Filtra pessoas com salário acima de 3000
dados_filtrados <- dados %>%
  filter(salario > 3000)

# Calcula a média de idade das pessoas filtradas
media_idade <- mean(dados_filtrados$idade)

# Exibe os dados filtrados e a média de idade
print(dados_filtrados)
```




Tip

```
# Verificando o diretório de trabalho  
getwd()
```

Usando o R Markdown *Prática*



Note

Crie um novo arquivo clicando em: **File > New File > R Markdown...**
Escolha um título, autor e selecione o formato desejado (HTML, PDF, Word).

- R Markdown é uma ferramenta que permite combinar **texto formatado** com **código executável**.
- É um documento dinâmico
- Ideal para **relatórios reprodutíveis, análises, artigos científicos e apresentações**.
- Usa o formato `.Rmd`.
- Para compilar, clique em Knit.

Estrutura básica de um arquivo R Markdown

```
---  
title: "Meu primeiro R Markdown"  
author: "Seu Nome"  
date: "2025-06-25"  
output: html_document  
---
```

Introdução

Texto explicativo aqui.

Códigos nas chunks
{r chunk do r}



Tip

Atalho para criar uma nova chunk: Ctrl + Alt + i

Principais componentes de um R Markdown

Elemento	Exemplo	Descrição
Texto	Markdown	Escrita formatada (títulos, listas, negrito, etc.)
Código R	{r plot(x)}	Código que será executado
Chunks	{r nome}	Blocos de código identificáveis
YAML Header	--- ... ---	Metadados sobre o documento



Tip

Use `echo = FALSE` para ocultar o código e mostrar apenas o resultado:

```
““{r echo=FALSE}
mean(c(1, 2, 3, 4, 5))
““
```

Use `eval = FALSE` para evitar a execução do código e mostrá-lo como exemplo:

```
““{r, echo=FALSE, eval=FALSE}
```

Você pode colocar textos, sem `#` para comentários:

Verificar outras funções para calcular a média...

Ou pode colocar códigos de análise, que você não quer usar no momento...

```
““
```



Tip

Use `message = FALSE`, `warning = FALSE` para esconder mensagens e avisos.

Confira os ‘Cheat Sheets’ para ver o que é possível fazer!

<https://posit.co/resources/cheatsheets/> (Diversos!)

□ [RMarkdown Cheatsheet](#) (abrir PDF)

Usando o R Markdown *Prática*



Tip

Se o objetivo é ‘costurar’ (criar) um arquivo html:

use ‘espaço + espaço + ’enter’ ou para quebrar linhas automaticamente

Se o arquivo de saída for .pdf, use

Quarto vs R Markdown

Característica	R Markdown (.Rmd)	Quarto (.qmd)
Engine	R (com suporte parcial a Python)	R, Python, Julia e Observable integrados
YAML	<code>output: html_document</code>	<code>format: html</code>
Execução de código	Integrada via knitr	Integrada via Jupyter ou knitr
Formatos de saída	HTML, PDF, Word, slides	HTML, PDF, Word, Reveal.js, livros, sites
Flexibilidade de linguagem	Limitada	Multilíngue e orientada a ciência de dados
Instalação necessária	R + RStudio	Pode usar com ou sem RStudio, CLI disponível
Renderização	Knit botão no RStudio	Render botão ou terminal com <code>quarto render</code>
Templates	Limitados, muitos voltados ao R	Amplo suporte a templates para blogs, sites, livros, etc.
Suporte para apresentações	Sim (com xaringan, ioslides, slidy, beamer)	Sim (nativamente com Reveal.js e Beamer)
Controle de layout	Limitado	Avançado com markdown, divs e CSS
Comunidade e futuro	Estável, porém sem grandes atualizações	Plataforma em crescimento e com maior integração

Utilizando a ferramenta “Projetos”

- Organiza seus diretórios
- Facilita a importação e exportação de bases de dados

- Facilita o compartilhamento de código

Prática

1. No canto superior direito, clique em:
 - **New Project**
 - **New Directory**
 - **New Project** novamente
2. Escolha o nome que desejar para o projeto e o local onde será salvo.
3. Clique em **Create Project** ☐

Tip

Geralmente não compensa salvar o *workspace* quando fechamos o Rstudio.
Para desabilitar essa opção, vá em *Tools > Global Options* e desmarque *Save workspace to RData* on exit:* Never*.

Utilizando o Git/GitHub

Git

- **Git** é um sistema de **controle de versões distribuído** criado por Linus Torvalds em 2005
- Permite o *versionamento do código*
 - Registro histórico de alterações em arquivos de código e projetos
 - Facilita o trabalho colaborativo, a recuperação de versões anteriores e o rastreamento de quem fez o quê
- Usado localmente no seu computador, e também com serviços como o GitHub

GitHub

- Plataforma de hospedagem de código e projetos que usam Git
- Não é exclusivo da linguagem R
- Facilita a **colaboração, rastreamento de mudanças e distribuição de projetos**
- Permite maior **transparência científica e reprodutibilidade**
- Permite **publicação de repositórios**: sites, documentos e dashboards

Fluxo básico de uso do GitHub

1. Criar conta no github.com
2. Criar novo repositório
3. Fazer *clone* no RStudio ou usar GitHub Desktop
4. Trabalhar localmente: editar scripts, salvar, testar
5. Realizar *commit* e *push* das alterações

Tip

Você também pode:

1. Clonar repositórios de outras pessoas (melhor para explorar, aprender...)
2. Fazer um 'fork' (permite sugerir mudanças)

Exemplo de estrutura de repositório

```
projeto-chikungunya/  
  dados/  
    base_limpa.xlsx  
  scripts/  
    analise_chikungunya.R  
  output/  
    grafico_incidentes.png  
  README.md  
  LICENSE
```

Crie um chunk de código, e no chatgpt, escreva:

“Abra o código abaixo no canvas, para edição.”

Utilizando IA como assistente de pesquisa

Dentre as formas de uso de IA dentro do RStudio, estão:

1. Co-piloto (GitHub Co-pilot)
2. Como um chat-bot, dentro do RStudio (pacote *chattr*)
3. Chat-bot externo (Microsoft Co-pilot, Chatgpt...)

Buscando ajuda (help!)

Algumas das principais formas de buscar ajuda são:

1. O *Help* ou documentação do R

```
help(sqrt)
```

2. Google
3. Foruns

1. *Stackoferflow* (<https://pt.stackoverflow.com>)
2. Posit Community (<https://forum.posit.co/>)
3. GitHub (<https://github.com/>)

4. IA

1. Microsoft co-pilot
2. Chat GPT
3. Gemini

4. Fundamentos da linguagem R

- O R é uma linguagem de programação dinâmica
- O R é uma linguagem de programação funcional
- Não é uma linguagem orientada a objetos, mas permite integração
- No R, também chamamos **objetos** as entidades que armazenam os dados
- O R é apenas uma linguagem. Quem faz os cálculos é o computador.

Fundamentos da linguagem R

Objetos

O R possui cinco classes básicas (atômicas) de objetos:

1. caracter
2. numérico (números reais)
3. inteiro
4. complexo
5. lógico (verdadeiro/falso)

Números

- No R, números são geralmente considerados como objetos numéricos (números reais com maior precisão ‘*double*’)
- Se quisermos usar números inteiros, podemos utilizar o sufixo L
 - P.e: $x \leftarrow c(2L, 4L)$
- Existe um número especial *Inf*, para representar o infinito.
 - P.e: $1/Inf$; 0
- *NaN* representa um valor indefinido (“*not a number*”).
 - P.e: $0/0$; NaN

Atributos

Os objetos do R podem ter atributos

- *names*, *dimnames*
- *dimensions*, para matrizes ou arranjos
- *class*
- *length*
- Outras, definidas pelo usuário

Podemos acessar os atributos de um objeto com a função *attributes()*

Entrando com os insumos

No *prompt* do R digitamos **expressões**.

O símbolo '`<-`' é o **operador de atribuição**.

```
> x <- 1
```

A **gramática da linguagem** determina se uma expressão está ou não completa.

```
> x <- ## expressão incompleta
```

```
# indica comentário. Tudo depois dele é ignorado.
```

Quando uma expressão completa é colocada no **prompt** ela é avaliada, e o resultado da avaliação é retornado.

```
x # print automático
```

```
print(x) # print explícito
```

```
# [1] 5 indica que x é um vetor, e que 5 é o primeiro elemento.
```

Tip

Podemos utilizar o operador '`:`' para criar inteiros em sequência.

```
x <- 1:22
```

```
x
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
```

Vetores

- O objeto mais básico é o **vetor**
- Os objetos mais elaborados são uma extensão de vetores
- Um vetor contém apenas objetos atômicos da mesma classe
- A função `c()` pode ser utilizada para criar vetores de objetos

```
x <- c(0.4, 0.9)           # numérico
x <- c(TRUE, FALSE)        # lógico
x <- c(T, F)               # lógico
x <- c("Vacinado", "Não Vacinado", "Dúvida") # caractere
x <- 3:25                  # inteiro
x <- c(2+0i, 1+4i)         # complexo
```

Tip

Utilize a função `class()` para verificar o tipo de objeto criado

```
class(x)
```

```
[1] "integer"
```

Ou observe a classe no painel superior direito.

Misturando objetos

O que acontece quando fazemos:

```
y <- c(1.7, "a") # caractere  
y <- c(TRUE, 2)  # numérico  
y <- c("a", TRUE) # caractere
```

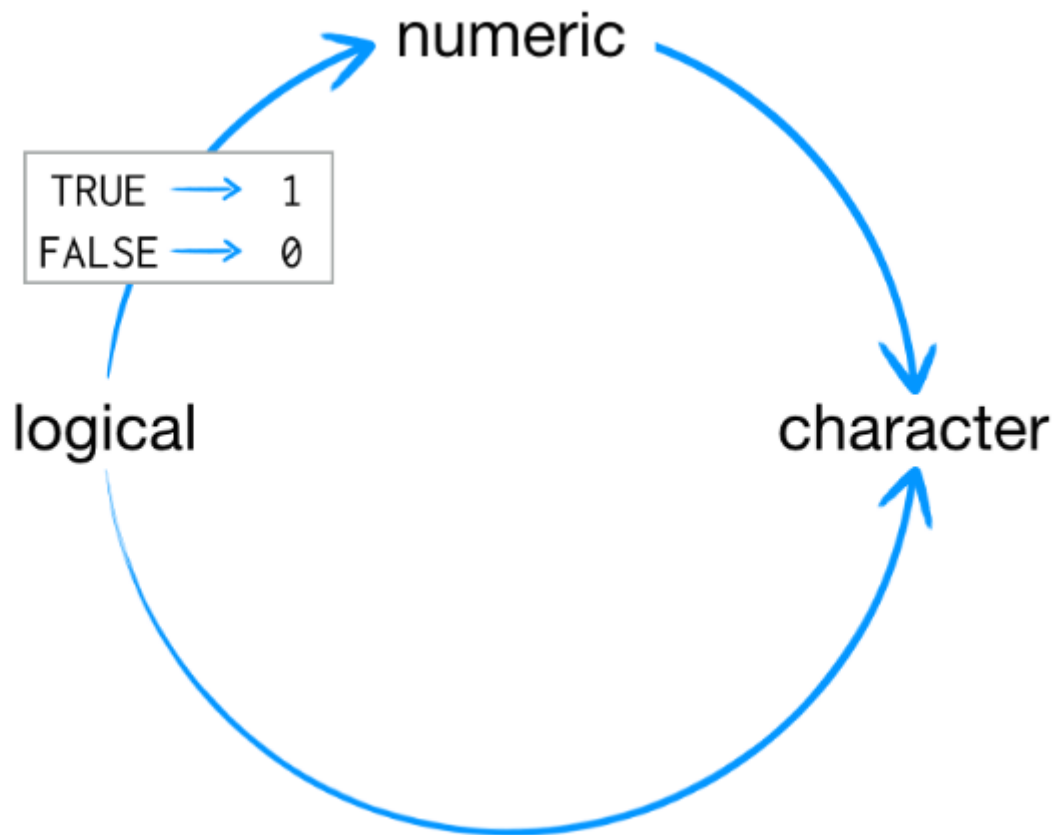


Figure 3: Regra de conversão. Fonte: Hands-on Programming with R, Grolemund (2015)

Conversão explícita

```
x <- 0:6  
class(x)  
[1] "integer"  
  
as.numeric(x)  
[1] 0 1 2 3 4 5 6  
  
as.logical(x)  
[1] FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
> as.character(x)
[1] "0" "1" "2" "3" "4" "5" "6"
```

Matrizes

- Matrizes são vetores, com o atributo *dimensão*
 - O próprio atributo *dimensão* é um vetor
- São construídas a partir da coluna (primeira à esquerda em diante)

```
m <- matrix(1:6, nrow = 2, ncol = 3)
m
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
dim(m)
```

```
[1] 2 3
```

```
attributes(m)
```

```
$dim
[1] 2 3
```

Tip

As funções `cbind()` e `rbind()` são muito comuns e podem ser utilizadas para criar matrizes:

```
x <- 1:4
y <- 10:13

cbind(x,y)
```

```
      x  y
[1,]  1 10
[2,]  2 11
[3,]  3 12
[4,]  4 13
```

```
rbind(x,y)
```

```
      [,1] [,2] [,3] [,4]
x         1    2    3    4
y        10   11   12   13
```

Listas

- Listas são um tipo especial de vetores

- podem conter elementos de diversas classes

```
x <- list(1, "a", TRUE, 1 + 4i)
```

```
x
```

```
[[1]]
```

```
[1] 1
```

```
[[2]]
```

```
[1] "a"
```

```
[[3]]
```

```
[1] TRUE
```

```
[[4]]
```

```
[1] 1+4i
```

Note

Nomes

Os objetos do R também podem ter *nomes* como atributo

- Facilita a organização o código
- Interessante para visualizações

Prática

```
x <- 1:3
```

```
names(x)
```

```
names(x) <- c("Hospital", "UBS", "S.Espec")
```

```
x
```

```
names(x)
```

Podemos criar listas já com o atributo nome

```
x <- list(Hosp = 1:4, UBS = 2, Espec = "Sim")
```

```
x
```

```
$Hosp
```

```
[1] 1 2 3 4
```

```
$UBS
```

```
[1] 2
```

```
$Espec
```

```
[1] "Sim"
```

Fatores

- São utilizados para representar dados categóricos

- Podem ser ordenados ou não
- Podem ser vistos como vetores inteiros que possuem um rótulo (*label*)
- São tratados de forma especial em modelagem
- A ordem dos níveis pode ser estabelecida com o argumento *levels* da função *factor()*.
 - Relevante para modelagem, pois o primeiro nível é utilizado como *baseline*

```
x <- factor(c("sim", "sim", "não", "sim", "não"))
x
```

```
[1] sim sim não sim não
Levels: não sim
```

```
table(x)
```

```
x
não sim
  2    3
```

```
x <- factor(c("sim", "sim", "não", "sim", "não"),
            levels = c("sim", "não"))
```

Fatores

Tipos de Variáveis

i Note

Classificação	Tipo	Tipo reconhecido pelo R	Exemplo de Variável	Valores
Qualitativa				
Nominais	Texto	<i>character</i>	Município	Rosas
Ordinais	Categorias	<i>factor</i>	Escolaridade	(0) Sem escolaridade (1) Fundamental 1º ciclo (2) Fundamental 2º ciclo (3) Médio (4) Superior (5) Não se aplica (9) Ignorado
Quantitativas				
Discretas	Números inteiros	<i>numeric</i> ou <i>integer</i>	Número de casos	125
Contínua	Números decimais	<i>numeric</i> ou <i>double</i>	Taxa de incidência	120.38

Figure 4: Fonte: Análise de dados para a vigilância em saúde (ABRASCO)

Valores ausentes (*missing values*)

- Valores ausentes são denotados por *NA* ou *NaN*
- *NaN* refere-se a operações matemáticas não definidas
 - um *NaN* é também um *NA*, mas o inverso não é verdadeiro
- Podemos usar *is.na()* para testar se um objeto é *NA*
- Podemos usar *is.nan()* para testar se um objeto é *NaN*

```
x <- c(1, 2, NA, 10, 3)
```

```
is.na(x)
```

```
[1] FALSE FALSE TRUE FALSE FALSE
```

```
is.nan(x)
```

```
[1] FALSE FALSE FALSE FALSE FALSE
```

Quadro de dados (*data frames*)

São utilizados para armazenar dados tabulares, tabelas

- São representados como um tipo especial de lista onde cada elemento da lista possui o mesmo comprimento
- Cada elemento da lista é como uma **variável**, e o comprimento de cada lista o *número de linhas*
- Possuem um atributo especial: *row.names*
- Geralmente são criados chamando *read.table()* ou *read.csv()*

Também podemos criá-los com a função *data.frame()*:

```
x <- data.frame(paciente = 1:4, retorno = c(T, T, F, F))
```

```
x
```

```
  paciente retorno
1         1    TRUE
2         2    TRUE
3         3   FALSE
4         4   FALSE
```

```
nrow(x)
```

```
[1] 4
```

```
ncol(x)
```

```
[1] 2
```

Data Frames

Subsetting

- Refere-se à operação de extrair subconjuntos dos objetos do R
- Existem formas diferentes, dependendo do tipo de objeto

Vetores

- Utilizamos []
- Retorna um subconjunto cujo elemento é da mesma classe do original
- Pode ser utilizado para selecionar mais de um elemento

Subsetting

Prática

```
x <- c("a", "b", "c", "c", "d", "a")
x[1]

x[2]

x[1:4]

x[x > "a"]
```

Vetores

Matrizes

- Utilizamos []
- Consideramos os índices i (linhas) e j (colunas): (i,j)

```
x <- matrix(1:6, 2, 3)
x[1,2]
```

```
[1] 3
```

```
x[2,1]
```

```
[1] 2
```

- Podemos selecionar apenas linhas ou colunas

```
x[1,]
```

```
[1] 1 3 5
```

```
x[, 2]
```

```
[1] 3 4
```

Subsetting

Listas

Podemos utilizar `[ ]`, `[ [ ] ]` ou `$`

```
x <- list(Hosp = 1:4, UBS = 2)
```

```
x[1]
```

```
$Hosp  
[1] 1 2 3 4
```

```
x[[1]]
```

```
[1] 1 2 3 4
```

```
x$UBS
```

```
[1] 2
```

```
x[["UBS"]]
```

```
[1] 2
```

Subsetting

Data frames

```
pacientes <- data.frame(  
  id = c(101, 102, 103, 104),  
  nome = c("Ana", "Carlos", "Beatriz", "Daniel"),  
  idade = c(35, 42, 29, 50),  
  hipertensao = c(TRUE, TRUE, FALSE, TRUE),  
  glicemia = c(92, 110, 85, 130)  
)  
  
# Selecionar a primeira linha, segunda coluna (nome do primeiro paciente)  
pacientes[1, 2]      # "Ana"  
  
# Selecionar as duas primeiras linhas, todas as colunas  
pacientes[1:2, ]  
  
# Selecionar todas as idades  
pacientes[, "idade"]  
  
# Selecionar pacientes com glicemia maior que 100 (potencial risco de diabetes)  
pacientes[pacientes$glicemia > 100, ]  
  
# Selecionar pacientes com hipertensão  
pacientes[pacientes$hipertensao == TRUE, ]
```

Subsetting

Estruturas de controle

- São mais úteis para escrever **programas** (*scripts*)
- Permitem controlar o fluxo de execução do *script*
- Na linha de comando, as funções *apply* são mais úteis

Estruturas de controle

Estruturas de controle:

- if, else
- for
- while
- repeat
- break
- next
- return

Estruturas de controle

- If, else: testam um condição

```
if(<condição>) {  
    ## faz algo  
} else {  
    ## faz outra coisa  
}
```

- ```
if(x > 3) {
 y <- 10
} else {
 y <- 0
}
```

Estruturas de controle

- for: executa um *loop* uma quantidade fixa de vezes  
*Prática*

```
for(i in 1:10) {
 print(i)
}
```

Estruturas de controle

- while: executa um *loop* enquanto uma condição for verdade

```
conte <- 0
while(conte < 10) {
 print(conte)
 conte <- conte + 1
}
```

Estruturas de controle

- repeat: executa um *loop* infinito
- break: termina a execução de um *loop*

```
x0 <- 1
tol <- 1e-8
repeat {
 x1 <- computeEstimate()
 if(abs(x1 - x0) < tol) {
 break
 } else {
 x0 <- x1
 }
}
```

## Estruturas de controle

### Note

1e-8 refere-se a *notação científica*  
 $1e-8 = 1 \times 10^{(-8)} = 0,00000008$   
 abs( ) é a função que calcula o módulo, ou valor absoluto

## Estruturas de controle

- next: pula interações de um *loop*

```
for(i in 1:100) {
 if(i <= 20) {
 ## Pula as primeiras 20 iterações
 next
 }
 ## Faz algo aqui...
}
```

- return: termina a execução de uma função

## Estruturas de controle

### Tip

Sempre que usar estruturas de controle, fique atento aos { }, que iniciam e fecham seu conteúdo.

## Estruturas de controle

## Funções

- São mais um tipo de objeto
  - No lugar de valores, contém código
  - O código é armazenado em um formato que permite seu uso facilmente, em novas situações
- São construídas estabelecendo-se seus **argumentos**
- A função *arg()* permite ver os argumentos de uma função *Prática*

```
Observando os argumentos de uma função
```

```
args(round)
```

- Entramos com os argumentos de uma função na ordem em que foram criados
- Geralmente os dois primeiros argumentos de uma função são conhecidos, então podemos omiti-los

```
round(x = 1.141598, digits = 2) # argumentos explícitos
```

```
round(1.141598, 2) # argumentos implícitos
```

## Funções



### Tip

Boa prática: escrever os nomes dos argumentos a partir do 3°.

## Funções

### Escrevendo suas próprias funções

- Passamos de ‘morador’ para ‘arquiteto’
- Uma função tem três componentes básicos

- 1) nome
- 2) corpo de código
- 3) conjunto de argumentos

Utilizamos a função *function()*

```
Prática
```

```
Exemplo de função criada pelo usuário:
```

```
joga_dados <- function() {
 dado <- 1:6
 dados <- sample(dado, size = 2, replace = TRUE)
 sum(dados)
}
```

```
Para chamarmos a função, utilizamos o nome atribuído a ela:
```

```
joga_dados()
```

```
[1] 7
```

```
Quando acionamos a função, o R executa o código e retorna o
resultado da última linha de código
```

## Escrevendo funções



### Tip

Com o curso no console, utilize ‘seta para cima’, para buscar os comandos executados anteriormente.



## Escrevendo funções

Podemos utilizar argumentos que ainda não tenham sido definidos.

Neste caso, eles precisam ter valores atribuídos quando chamamos a função.

```
Prática
```

```
joga_dados <- function(dado) {
 dados <- sample(dado, size = 2, replace = TRUE)
 sum(dados)
}
```

```
joga_dados(dado = 1:4)
```

```
[1] 2
```

## Escrevendo funções

Podemos atribuir valores iniciais ao argumento:

```
Prática
```

```
joga_dados <- function(dado = 1:6) {
 dados <- sample(dado, size = 2, replace = TRUE)
 sum(dados)
}
```

```
joga_dados()
```

```
joga_dados(dado = 1:12)
```

## Escrevendo funções

## 5. Lendo os dados *Prática*

### Lendo os dados .csv

```
if(!require(readr)) install.packages("readr")
library(readr)

dados <- read_csv2(file = "e_sus_notifica.csv")
```

### Lendo dados .dbf

```
if(!require(foreign)) install.packages("foreign")
library(foreign)

dados_sinan <- read.dbf(file = 'NINDINET.dbf', as.is = TRUE)

head(dados_sinan)
```

### Lendo dados .xlsx

```
if(!require(readxl)) install.packages("readxl")
library(readxl)

dados_sivep <- read_excel("sivep_gripe.xlsx",
 sheet = "SIVEPGRIPE",
 skip = 0)
```

## 6. Manipulando dados

- É parte da pesquisa quantitativa que toma mais tempo
- Também é onde acontece a maior parte dos erros
- Quais os problemas de se fazer isso em planilhas eletrônicas? ☐



Tip

Boa prática: indicar todos os pacotes que serão utilizados no começo do script.

```
if(!require(tidyverse)) install.packages("tidyverse");library(tidyverse)
```

Carregando pacotes exigidos: tidyverse

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr 1.1.4 v readr 2.1.5
v forcats 1.0.0 v stringr 1.5.1
v ggplot2 3.5.2 v tibble 3.2.1
v lubridate 1.9.4 v tidyr 1.3.1
v purrr 1.0.4
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
if(!require(foreign)) install.packages("foreign");library(foreign)
```

Carregando pacotes exigidos: foreign

```
if(!require(readxl)) install.packages("readxl");library(readxl)
```

Carregando pacotes exigidos: readxl

```
if(!require(readr)) install.packages("readr");library(readr)
if(!require(janitor)) install.packages("janitor");library(janitor)
```

Carregando pacotes exigidos: janitor

Anexando pacote: 'janitor'

Os seguintes objetos são mascarados por 'package:stats':

chisq.test, fisher.test

```
if(!require(skimr)) install.packages("skimr");library(skimr)
```

Carregando pacotes exigidos: skimr

```
if(!require(stringr)) install.packages("stringr");library(stringr)
if(!require(stringi)) install.packages("stringi");library(stringi)
```

Carregando pacotes exigidos: stringi

```
if(!require(lubridate)) install.packages("lubridate");library(lubridate)
```

Manipulando dados



Tip

Organize os dados do seu projeto dentro de uma pasta separada, p.e., “Dados”

## Lendo os arquivos necessários

```
base <- read.dbf(file = 'Dados/NINDINET.dbf', as.is = TRUE)

CID10 <- read_csv2("Dados/CID-10-CATEGORIAS.CSV",
 locale = locale(encoding = "latin1"))
```

i Using "','" as decimal and "'.'" as grouping mark. Use `read\_delim()` for more control.

Rows: 2045 Columns: 6

-- Column specification -----

Delimiter: ";"

chr (5): CAT, CLASSIF, DESCRICAO, DESCRABREV, REFER

lgl (1): EXCLUIDOS

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
dados <- read_csv(file = "Dados/cobertura_hepatiteb_rosas_2016_2020_A.csv")
```

Rows: 2 Columns: 6

-- Column specification -----

Delimiter: ","

chr (6): Municipio, Ano 2016, Ano 2017, Ano 2018, Ano 2019, Ano 2020

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
eapv_2021m <- read_xlsx('Dados/notificacao_eapv_2021m.xlsx')
```

```
tabela_municipios <- read_excel("Dados/tabela_municipios.xlsx",
 sheet = 1,
 skip = 0)
```



Tip

Para visualizar os tópicos de um arquivo Rmarkdown/Quarto, use ‘Alt’ + ‘o’.



Tip

Para examinar o código de uma *chunk*, digite ‘Alt’ + ‘e’.

O argumento **as.is**:

- Evita que texto sejam transformadas em fatores de imediato
- Isso torna a manipulação mais flexível (ex.: limpeza de dados, padronização de strings, etc)
- Fatores consomem mais memória. Relevante em arquivos grandes

Manipulando dados



Tip

Use ‘Ctrl’ + F para buscar dentro do seu documento R.

Use ‘Replace’ para substituir palavras.

Muito útil quando queremos alterar o comportamento de diversas chunks ao mesmo tempo.

Manipulando dados

## Visualização inicial dos dados

```
head(base)

library(tidyverse)
glimpse(base)
```



Tip

Execute o código da chunk com o ‘play’, no canto superior direito.

Se uma chunk de código possui diversas saídas (visualizações), clique nos quadros do rstudio abaixo da chunk para visualizar todas as saídas.

## Selecionando variáveis (colunas)

```
base_menor <- select(base, DT_NOTIFIC, DT_NASC, CS_SEXO, CS_RACA, ID_MN_RESI,
 ID_AGRAVO)
```

## Piping

O R nos permite utilizar *piping* (encadeamento) para ligar funções.

```
base_menor <- base |>
 select(DT_NOTIFIC, DT_NASC, CS_SEXO, CS_RACA, ID_MN_RESI, ID_AGRAVO)
```

- O primeiro argumento da função é passado de antemão
- Operador de ‘pipe’ antigo: %>%
- Operador do tidyverse: |>

Incluindo mais uma função no pipe

```
base |>
 select(DT_NOTIFIC, DT_NASC, CS_SEXO, CS_RACA, ID_MN_RESI, ID_AGRAVO) |>
 head()
```

Piping

## Excluindo colunas

```
base_menor |>
 select(-DT_NASC) |>
 head()
```

## Criando colunas

```
Criando uma nova variável, tempo de digitação:
base_menor_2 <- base |>
select(NU_NOTIFIC, ID_AGRAVO, DT_NOTIFIC, DT_DIGITA) |>
 mutate(TEMPO_DIGITA = DT_DIGITA - DT_NOTIFIC)

Visualizando a var DT_DIGITA como numérica
base_menor_2 |>
 mutate(TEMPO_DIGITA = as.numeric(TEMPO_DIGITA)) |>
 head()
```

## Limpando caracteres e padronizando as colunas

```
Trabalhando com o banco de dados { `CID-10-CATEGORIAS.CSV` }

Utilizando a função `colnames()` para checar as variáveis
colnames(cid10_categorias)

Utilizando a função `clean_names()` (janitor) para editar o nome das variáveis
cid10_categorias_nova <- clean_names(cid10_categorias)

Visualizando as variáveis após a transformação
colnames(cid10_categorias_nova)
```



### Tip

Utilize `'pacote :: funcao'` para especificar de qual pacote vem a função.  
Muito útil quando uma função existe em pacotes diferentes.

```
Utilizando a função `clean_names()` (janitor) para editar o nome das variáveis
cid10_categorias_nova <- janitor::clean_names(cid10_categorias)

names(cid10_categorias_nova) # A função 'names()' é mais geral que a 'colnames()'.
```

Limpando caracteres e padronizando as colunas

## Filtrando colunas

```
Filtrando os CIDs igual a B19, da coluna "ID_AGRAVO"
com tempo de digitação maior que sete dias
base_menor_2 |>
 filter(ID_AGRAVO == "B19", TEMPO_DIGITA > 7) |>
 head()

Filtrando os registros iguais a B19 OU A279 OU B54 na coluna "ID_AGRAVO"
base_menor_2 |>
 filter(ID_AGRAVO == "B19" | ID_AGRAVO == "A279" | ID_AGRAVO == "B54") |>
 head(20)

Fazendo o mesmo filtro utilizando o operador `%in%`
base_menor_2 |>
 filter(ID_AGRAVO %in% c("B19", "A279", "B54")) |>
 head(20)
```

## Sumarizando colunas

```
Agrupando as notificações pelos agravos e contando a frequência de notificações
base_menor_2 |>
 group_by(ID_AGRAVO) |>
 count(ID_AGRAVO) |>
 print(n = 20)
```

```
base_menor_2 |>

Agrupando as notificações pelos agravos
group_by(ID_AGRAVO) |>

Utilizando a função `summarise()` para criar novas colunas de síntese
summarise(

 # Criando uma coluna de total, utilizando a função `n()`
 total_agravos = n(),

 # Criando uma coluna de média, utilizando a função `mean()`
```

```
media_digita = mean(TEMPO_DIGITA)
)
```

Sumarizando colunas

## Modificando a ordem das colunas

```
base_menor_2 |>

Agrupando as notificações pelos agravos
group_by(ID_AGRAVO) |>

Utilizando a função `summarise()` para criar novas colunas de síntese
summarise(total_agravos = n(),
 media_digita = mean(TEMPO_DIGITA)) |>

Ordenando a tabela pela ordem decrescente da média de tempo de digitação
arrange(desc(media_digita))
```

## Avaliando a completude dos dados

```
base |>

Utilizando a função `summarise()` para criar novas colunas de síntese
summarise(

 # Criando uma coluna com a soma de todos os registros da variável raça/cor devidamente preenchidos
 total_completo = sum(!is.na(CS_RACA)),

 # Criando uma coluna com o total de registros na tabela
 total_registros = n(),

 # Criando uma coluna com a soma de registros com a variável raça/cor em branco
 total_missing_raca = sum(is.na(CS_RACA)),

 # Criando uma coluna de porcentagem de completude (preenchimento)
 taxa_completude = (total_completo / total_registros) * 100
)
```

### **i** Note

Em programação, “!” é utilizado para negar uma expressão lógica.

## Renomeando valores

```
base |>

Contabilizando o número de registros conforme as categorias de preenchimento
```



```
da variável "CS_RACA"
count(CS_RACA)

base |>

Transformando os valores em branco em 9 na coluna "CS_RACA"
mutate(CS_RACA = replace_na(CS_RACA, replace = 9)) |>

Contabilizando o número de registros conforme as categorias de preenchimento
da variável "CS_RACA"
count(CS_RACA)
```

## Transformando os dados no formato largo em formato longo

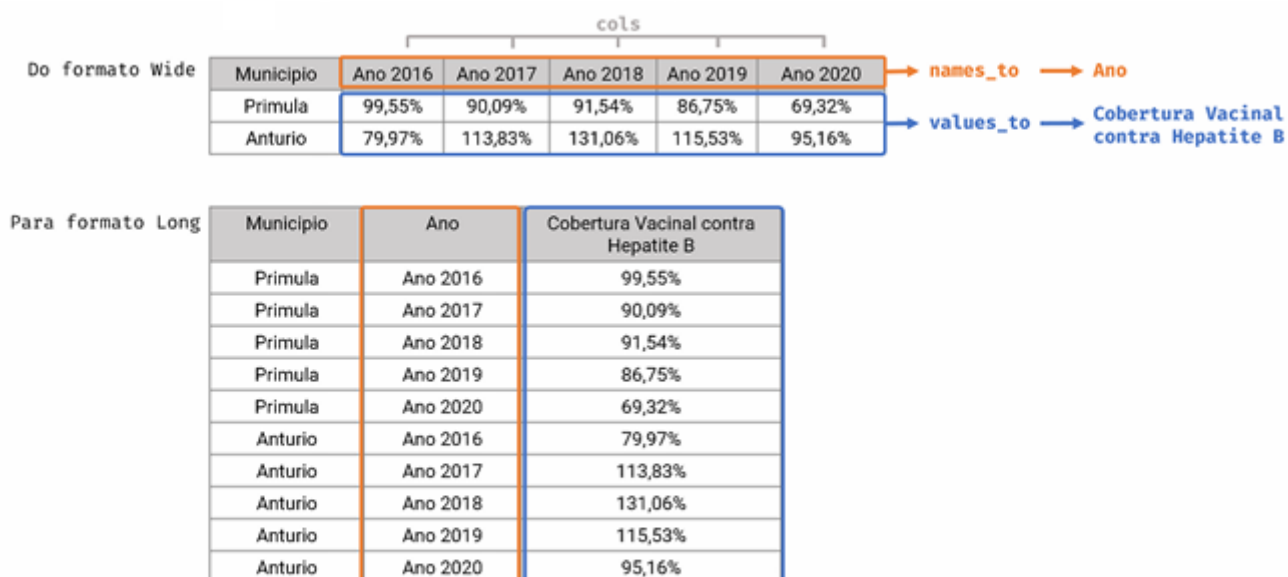


Figure 5: Fonte: Análise de dados para vigilância em saúde (ABRASCO)

```
Trabalhando com o banco de dados `cobertura_hepatiteb_rosas_2016_2020_A.csv`

Criando o objeto {`dados_longos`}
dados_longos <- dados |>

Utilizando a função `pivot_longer()` para transformação de colunas
pivot_longer(

Definindo as colunas que serão transformadas
cols = c("Ano 2016", "Ano 2017", "Ano 2018", "Ano 2019", "Ano 2020"),

Definindo o nome da variável nova que receberá os nomes acima
```

```

names_to = "Ano",

Definindo o nome da variável nova que receberá os valores da tabela
values_to = "Cobertura Vacinal contra Hepatite B",

Retirando a palavra "Ano " antes de cada valor da variável Ano
Também estamos retirando o espaço depois da palavra "Ano"
names_prefix = "Ano "
)
Visualizando a tabela {`dados_longos`} no formato longo
dados_longos

```

## Transformando os dados do formato longo em formato largo

```

Criando o objeto {`dados_largos`}
dados_largos <- dados_longos |>

Utilizando a função `pivot_wider()` para transformação de colunas
pivot_wider(

Definindo de qual variável estamos resgatando os nomes das colunas
names_from = "Ano",

Definindo de qual variável estamos resgatando os valores das colunas
values_from = "Cobertura Vacinal contra Hepatite B")

visualizando a tabela
dados_largos

```

## Separando o conteúdo de uma variável em várias colunas

```

Trabalhando com o banco de dados `notificacao_eapv_2021m.xlsx`

Visualizando a tabela {`eapv_2021m`} com a função `View()`
View(eapv_2021m)

Selecionando e visualizando variáveis específicas
eapv_2021m |>

Selecionando três colunas do data.frame {`eapv_2021m`}
select(imunobiologico_vacina, dose, data_da_aplicacao) |>

Utilizando a função `head()` para visualizar as primeiras linhas da tabela
head()

```

```

eapv_2021m |>
Dividindo a coluna `imunobiologico_vacina` em três novas colunas
separate(
Definindo a coluna que será separada
col = imunobiologico_vacina,

```

```
Definindo os nomes das novas colunas
into = c("vac_event_1", "vac_event_2", "vac_event_3"),
Definindo qual o caractere que está sendo utilizado dentro das colunas
sep = "\\|"
) |>
Selecionando as novas colunas para visualização
select("vac_event_1", "vac_event_2", "vac_event_3")
```

Separando o conteúdo de uma variável em várias colunas

## Recodificando linhas e colunas

```
base_menor |>
Renomeando os valores da variável CS_SEXO usando a função `mutate()` e `case_when()`
mutate(
 sexo_cat = case_when(
 CS_SEXO == "M" ~ "Masculino",
 CS_SEXO == "F" ~ "Feminino",
 CS_SEXO == "I" | is.na(CS_SEXO) ~ "Ignorado",
 TRUE ~ NA_character_
)
) |>
Visualizando a tabela {`base_menor`} recodificada
head()
```

```
base_menor <- base_menor |>
mutate(
 raca_cor_cat = case_when(
 # Se o valor da coluna for igual a "1" transforme para "Branca"
 CS_RACA == "1" ~ "Branca",
 # Se o valor da coluna for igual a "2" transforme para "Preta"
 CS_RACA == "2" ~ "Preta",
 # Se o valor da coluna for igual a "3" transforme para "Amarela"
 CS_RACA == "3" ~ "Amarela",
 # Se o valor da coluna for igual a "4" transforme para "Parda"
 CS_RACA == "4" ~ "Parda",
 CS_RACA == "5" ~ "Indígena",
 CS_RACA == "9" | is.na(CS_RACA) ~ "Ignorado",
 TRUE ~ NA_character_
)
) |>
head()
```

## Recodificando linhas e colunas

```
base |>
Utilizando a função `mutate()` para criar colunas
mutate(
 # Criando uma coluna de idade conforme a codificação da variável NU_IDADE_N
 idade_anos = if_else(str_sub(NU_IDADE_N, start = 1, end = 1) == "4",
 as.numeric(str_sub(NU_IDADE_N, start = 2, end = 4)), 0),
 # Criando uma coluna de faixa etária a partir da variável idade dos casos notificados
```

```
utilizando a função `cut()`
fx_etaria = cut(
 # Definindo qual variável será classificada em faixas
 x = idade_anos,
 # Definindo os pontos de corte das classes
 breaks = c(0, 10, 20, 60, Inf),
 # Definindo os rótulos das classes
 # Definindo o tipo do ponto de corte
 right = FALSE
)
) |>
select(NU_IDADE_N, idade_anos, fx_etaria, everything()) # Muda ordem de visualização
```

Recodificando linhas e colunas



Tip

Comentários do meio da linha podem ser úteis, mas podem dificultar a visualização do código.

```
Observe o mesmo código, sem comentários.

base |>
 mutate(
 idade_anos = if_else(str_sub(NU_IDADE_N, 1, 1) == "4",
 as.numeric(str_sub(NU_IDADE_N, 2, 4)), 0),

 fx_etaria = cut(
 x = idade_anos,
 breaks = c(0, 10, 20, 60, Inf),
 right = FALSE
)
) |>
 select(NU_IDADE_N, idade_anos, fx_etaria, everything())
```

## Unindo tabelas (bases de dados)

| Função                    | Descrição                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>inner_join()</code> | Retorna apenas as observações que têm correspondência nas duas tabelas.                                         |
| <code>left_join()</code>  | Retorna <b>todas</b> as observações da tabela da <b>esquerda</b> , com correspondências da direita (se houver). |
| <code>right_join()</code> | Retorna <b>todas</b> as observações da tabela da <b>direita</b> , com correspondências da esquerda.             |
| <code>full_join()</code>  | Retorna <b>todas</b> as observações de ambas as tabelas (união completa).                                       |

```
Verificando o tipo de variável na coluna `ID_MN_RESI` do NINDINET

class(base_menor$ID_MN_RESI)

Vamos utilizar o banco 'tabela_municípios.xlsx'.
```

```
class(tabela_municipios$ID_MUNICIPIO)

Transformando apenas a variável `ID_MUNICIPIO` do data.frame {`tabela_municipios`}
em 'integer'

tabela_municipios$ID_MUNICIPIO <- as.integer(tabela_municipios$ID_MUNICIPIO)

left_join(
 # Unindo a tabela {`base_menor`}
 x = base_menor,
 # com a tabela {`tabela_municipios`}
 y = tabela_municipios,
 # Selecionando as colunas `ID_MN_RESI` e `ID_MUNICIPIO` para unir os bancos de dados
 by = c("ID_MN_RESI" = "ID_MUNICIPIO")) |>
 # Visualizando a união realizada
head()
```

Unindo tabelas

## 7. Lidando com textos

### Contando caracteres e extraindo textos

```
Criando um novo objeto com as seleções feitas (dados CID-10-CATEGORIAS.CSV)

nomes_cid <- CID10 |>
 # Selecionar apenas as linhas de interesse (linhas 1 a 15)
 slice(1:15) |>
 # Selecionando apenas a variável "DESCRICAO" e transformando em `character` simples
 pull(DESCRICAO)

nomes_cid

Contando quantos caracteres existem em cada linha
str_length(nomes_cid)

Extraindo caracteres entre a 1ª e a 11ª posição da linha
str_sub(nomes_cid, start = 1, end = 11)

Indicando a posição final da primeira palavra de cada texto
str_sub(nomes_cid,
 start = 1,
 end = c(6,6,6,10,6,6,8,6,9,8,11,11,11,11,11))

Extraindo a primeira palavra
word(nomes_cid, 1, sep = " ")
```

### Alterando a ordem e reordenando texto

```
Ordenando o objeto {`nomes_cid`} para o formato ascendente (A-Z)
str_sort(nomes_cid)

Ordenando o objeto {`nomes_cid`} para o formato decrescente (Z-A)
str_sort(nomes_cid, decreasing = TRUE)
```

### Unindo *strings*

```
Unindo strings para a formação da palavra "Tuberculose"
str_c("Tuber","culo","se")

Unindo strings incluindo espaço entre elas
adicionaremos o argumento `sep = " "`
str_c("Tuberculose","respiratória","com","confirmação",
 "bacteriológica","e histológica", sep = " ")

Salvando as modificações no objeto {`agravo`}
agravo <- c("Tuberculose","respiratória","com","confirmação",
 "bacteriológica","e histológica")
```

```
Unindo **strings** armazenadas no objeto {`agravo`}
em um único *string* de texto
str_c(agravo, collapse = " ")
```

## Transformando *strings*

```
Transformando todos as letras em maiúsculas
str_to_upper(nomes_cid)

Transformando todos as letras em minúsculas
str_to_lower(nomes_cid)

Transformando apenas a primeira maiúscula e restante das palavras em minúsculas
str_to_sentence(nomes_cid)

Padronizando a escrita (pacote stringi)

Vetor com diferentes grafias

nomes <- c("infecções", "infeccções", "infecções", "infeccoes")

Transformação das letras especiais ou com acentuação
stri_trans_general(nomes, id = "Latin-ASCII")
```

## Identificando padrões em *strings*

```
Extraíndo linhas onde a palavra "Tuberculose" aparece
str_subset(nomes_cid, pattern = "Tuberculose")

Extraíndo linhas onde a palavra "tuberculose" aparece, independente se
maiúscula ou minúscula
str_subset(nomes_cid, pattern = fixed("tuberculose", ignore_case = TRUE))
```

---

```
Encontrando padrões no início do texto

CID10 |>
 filter(
 str_starts(
 # Utilizando a função `filter()` com a função `str_starts()`
 # Selecionando a variável usada para encontrar palavras no início da frase
 string = DESCRICAO,
 # Definindo a palavra a ser pesquisada e ignorando se maiúscula ou minúscula
 pattern = fixed("outras", ignore_case = TRUE)
)
) |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(CAT, DESCRICAO)
```

---

```
Encontrando padrões no final do texto
```

```
CID10 |>
Utilizando a função `filter()` com a função `str_ends()`
filter(
 str_ends(
 # Definindo a variável usada para encontrar palavras no final da frase
 string = DESCRICAO,
 # Definindo a palavra a ser pesquisada e ignorando se maiúscula ou minúscula
 pattern = fixed("bacterianas", ignore_case =
 TRUE)
)
) |>
Selecionando as variáveis que queremos utilizar com a função `select()`
select(CAT, DESCRICAO)
```

## Substituindo valores de texto

```
Criando um vetor de texto
agravo <- "Tuberculose respiratória, com confirmação bacteriológica e histológica"

Substituindo palavras
str_replace(agravo, pattern = "com", replacement = "sem")

Criando um vetor de texto
agravo <- "Tuberculose respiratória, com confirmação bacteriológica e histológica"

Substituindo palavras
str_replace_all(agravo, pattern = "ó", replacement = "o")
```



## 8. Organizando os eventos no tempo

### Transformando datas

```
Transformando uma string em data
data_1 <- as_date("2022-05-19")
Visualizando a string transformada em data
data_1

Verificando a classe do objeto `data_1`
class(data_1)

Transformando uma string em data sem definir o formato
as_date("19/05/2022")

Transformando uma string em data definindo o formato
as_date("19/05/2022", format = "%d/%m/%Y")

Transformando uma string do tipo das datas do SIM definindo o formato
as_date("01031998", format = "%d%m%Y")
```

### Cálculo com datas

```
dt_notific <- read.dbf(file = 'Dados/NINDINET.dbf') |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC, DT_SIN_PRI, DT_NASC) |>
 # Selecionar apenas as linhas de interesse (linhas 1 a 10)
 slice(1:10)

dt_notific
```

---

```
Somando 60 dias às datas de notificação de casos da tabela `{dt_notific}`
dt_notific$DT_NOTIFIC + 60

Subtraindo 60 dias as datas de notificação
#de casos à tabela `{dt_notific}`
dt_notific$DT_NOTIFIC - 60

Criando a tabela `{dt_notific_2}`
dt_notific_2 <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC) |>
 # Utilizando a função `mutate()` para criar a nova coluna "prazo_encerramento"
 mutate(prazo_encerramento = DT_NOTIFIC + 60)

dt_notific_2
```

### Cálculo com datas

---

```
Criando a tabela com a diferença entre 'DT_NOTIFIC' e 'DT_SIN_PRI'

dif_tempo <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC, DT_SIN_PRI) |>
 # Utilizando a função `mutate()` para criar a nova coluna "DIFERENCA"
 mutate(DIFERENCA = DT_NOTIFIC - DT_SIN_PRI)

Obtendo o resultado como inteiro
dif_tempo_2 <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC, DT_SIN_PRI) |>
 # Utilizando a função `mutate()` para criar a nova coluna "Diferenca"
 mutate(Diferenca = as.integer(DT_NOTIFIC - DT_SIN_PRI))

dif_tempo_2
```

## Cálculo com datas

---

```
Encontrando a idade em anos, com arredondamento

idade <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NASC, DT_SIN_PRI) |>
 # Utilizando a função `mutate()` para criar as novas colunas de idade
 mutate(
 IDADE_DIAS = as.integer(DT_SIN_PRI - DT_NASC),
 IDADE_ANOS = floor(IDADE_DIAS / 365.25)
)
```

## Cálculo com datas

## Extrações de elementos de data

```
Extrai o ano (lubridate)
year(dt_notific$DT_NOTIFIC)

Extrai o mês
month(dt_notific$DT_NOTIFIC)

Extrai o dia
day(dt_notific$DT_NOTIFIC)

Extrai o dia da semana
wday(dt_notific$DT_NOTIFIC)

Com o dia da semana abriado
wday(dt_notific$DT_NOTIFIC, label = TRUE)
```

```
Nome do dia da semana, sem abreviação
wday(dt_notific$DT_NOTIFIC, label = TRUE, abbr = FALSE)
```

---

```
Identificando a semana epidemiológica nas datas

Transformando todas as datas da variável `DT_NOTIFIC`
da tabela {`dt_notific`} em semana epidemiológica
com a função `epiweek()`
epiweek(dt_notific$DT_NOTIFIC)

Alterando a tabela {`dt_notific_2`}
dt_notific_2 <- dt_notific |>
 # Utilizando a função `mutate()` para criar novas colunas
 mutate(
 # Criando variável ano a partir da data de notificação
 # utilizando a função `year()`
 ano = year(DT_NOTIFIC),
 # Criando variável mes a partir da data de notificação
 # utilizando a função `month()`
 mes = month(DT_NOTIFIC),
 # Criando variável dia a partir da data de notificação
 # utilizando a função `day()`
 dia = day(DT_NOTIFIC),
 # Criando variável semana_num a partir da data de notificação
 # utilizando a função `wday()`
 semana_num = wday(DT_NOTIFIC),
 # Criando variável semana_nome a partir da data de notificação
 # utilizando a função `wday()`
 semana_nome = wday(DT_NOTIFIC, label = TRUE),
 # Criando variável semana_nome_completo a partir da data de notificação
 # utilizando a função `wday()`
 semana_nome_completo = wday(DT_NOTIFIC, label = TRUE, abbr = FALSE),
 # Criando variável semana_epidemiológica a partir da data de notificação
 # utilizando a função `epiweek()`
 semana_epidemiologica = epiweek(DT_NOTIFIC)
)

dt_notific_2
```

## Cálculo com datas

---

```
Agregando e indicando a data de final da semana epidemiológica

Criando a tabela {`dt_notific_5`}
dt_notific_5 <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC) |>
 # Utilizando a função `mutate()` para criar as novas variáveis
```

```

mutate(
 dia_semana = wday(DT_NOTIFIC),
 dif_dia_semana = 7 - dia_semana,
 DT_semana_epi = DT_NOTIFIC + dif_dia_semana
)

dt_notific_5

Outra forma

Criando a tabela {`dt_notific_6`}
dt_notific_6 <- dt_notific |>
 # Selecionando as variáveis que queremos utilizar com a função `select()`
 select(DT_NOTIFIC) |>
 # Utilizando a função `mutate()` para criar a nova coluna
 mutate(DT_semana_epi = DT_NOTIFIC + 7 - wday(DT_NOTIFIC))

dt_notific_6

```

Cálculo com datas

## Sequência de datas

```

Criando uma sequência de datas, criando dias de uma semana
as.Date("2022-01-01") + 0:7

Criando uma sequência de datas
seq.Date(from = as.Date("2022-01-01"), to = as.Date("2022-02-01"), by = "day")

```

## 9. Exportação dos dados

```
Salvando a tabela {base_menor} em seu computador na pasta "Meus documentos"
write_csv(x = base_menor,
 file = "C:/Users/UFR/Documents/Curso_CDPS/Dados/base_menor_curso.csv")

install.packages("writexl");library(writexl)
Salvando a tabela {base_menor} em seu computador na pasta "Meus documentos"
write_xlsx(x = base_menor,
 path = "C:/Users/UFR/Documents/Curso_CDPS/Dados/base_menor_curso/Dados/base_menor_curso.xlsx")
```

## 10. Referências

### Referências Principais

ABRASCO, **Análise de dado para Vigilância em Saúde - Curso Básico**, 2022.  
Módulos 1 a 3.

Wickham, H.; Cetinkaya-Rundel, M.; Golemund, G. **R for data science**, 2023.  
<https://r4ds.hadley.nz/>

Golemund, G. **Hands-on programming with R**, 2014.  
<https://rstudio-education.github.io/hopr/>

Peng, R. D. **R Programming for Data Science**, 2015.

### Outras Referências

*Análises Ecológicas no R*  
<https://analises-ecologicas.netlify.app/index.html>

*Ciência de Dados em R*  
<https://livro.curso-r.com/index.html>

*Ciência de Dados em R*  
<https://livro.curso-r.com/index.html>

*Zen do R*  
<https://curso-r.github.io/zen-do-r/>

*Introdução à Linguagem R: seus fundamentos e sua prática*  
[https://pedropark99.github.io/Introducao\\_R](https://pedropark99.github.io/Introducao_R)

*Ciência de Dados em R*  
<https://livro.curso-r.com/index.html>

*Análises Ecológicas no R*  
<https://analises-ecologicas.netlify.app>

*Introdução à Linguagem R: seus fundamentos e sua prática*  
[https://pedropark99.github.io/Introducao\\_R](https://pedropark99.github.io/Introducao_R)

*Fundamentos de Estatística para Ciência de Dados*  
<http://filipezabala.com/fdepcdd/>

*Introdução à Ciência de Dados*  
<https://rafalab.github.io/dslivro/>

*Happy Git and GitHub for the useR*  
<https://happygitwithr.com/>

*R Cookbook*  
<https://rc2e.com/>